Clarify the behavior of floating-point overflow

Document number:
P3899R1

Date:
2026-02-20

Audience:
SG6, EWG, CWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:
Jan Schultke <janschultke@gmail.com>
Matthias Kretz <m.kretz@gsi.de>

GitHub Issue:
wg21.link/P3899/github

Source:
github.com/eisenwave/cpp-proposals/blob/master/src/clarify-fp-overflow.cow

---

ABSTRACT: The current specification of floating-point overflow is unclear. This paper fixes it.

Contents

1 **Revision history**
1.1 Changes since R0

2 **Introduction**
2.1 Wording disputes
2.1.1 Undefined behavior on floating-point overflow
2.1.2 Undefined behavior on infinity propagation
2.1.3 Undefined behavior on yielding infinity
2.1.4 Conclusion
2.2 Constant expressions in C
2.3 Implementation divergence

3 **Design**
3.1 Floating-point overflow without infinity
3.2 Floating-point overflow with infinity
3.3 Floating-point underflow
3.4 Infinity propagation
3.5 Division by zero
3.6 NaN

4 **Impact on existing code**

5 **Implementation experience**

6 **Wording**
6.1 [basic.fundamental]
6.2 [expr.pre]
6.3 [expr.const]

7 **References**

1. Revision history

1.1. Changes since R0

- Reworded §2.1.3. Undefined behavior on yielding infinity
- Investigated floating-point underflow in §2.3. Implementation divergence
- Decided to allow underflow in constant expressions; see §3.3. Floating-point underflow
- Introduced the phrase "mathematically defined in the domain of real number arithmetic" in §6. Wording
- Rebased §6. Wording on N5032

2. Introduction

The current specification of floating-point overflow is unclear. Floating-point overflow occurs when finite operands are used as operands, and the result cannot be represented as a finite value. For [ISO/IEC 60559:2020] floating-point numbers, this results in infinity.

2.1. Wording disputes

Extensive discussion in CWG has shown that we cannot find consensus on what the current behavior is:

- Is overflow undefined behavior, such as in `FLT_MAX * 2`?
- Is infinity arithmetic undefined behavior, such as in `infinity() + 1`?
- Is merely yielding infinity undefined behavior, such as in `numeric_limits<T>::infinity()`?

See below a summary of each wording dispute.

2.1.1. Undefined behavior on floating-point overflow

The source of undefined behavior is located in [expr.pre] paragraph 4:

> If during the evaluation of an expression, the result is not mathematically defined or not in the range of representable values for its type, the behavior is undefined.
>
> [*Note*: Treatment of division by zero, forming a remainder using a zero divisor, and all floating-point exceptions varies among machines, and is sometimes adjustable by a library function. — *end note*]

An operation between two finite numbers is mathematically defined, except for division by zero (which is explicitly and unambiguously undefined according to [expr.mul]). However, the operation's result may not lie in the "range of representable values", and so it could have undefined behavior. A description of the range has been added recently by [CWG2723] to [basic.fundamental] paragraph 13:

> The minimum range of representable values for a floating-point type is the most negative finite floating-point number representable in that type through the most positive finite floating-point number representable in that type. In addition, if negative infinity is representable in a type, the range of that type is extended to all negative real numbers; likewise, if positive infinity is representable in a type, the range of that type is extended to all positive real numbers.
>
> [*Note*: Since negative and positive infinity are representable in ISO/IEC/IEEE 60559 formats, all real numbers lie within the range of representable values of a floating-point type adhering to ISO/IEC/IEEE 60559. — *end note*]

This wording has been interpreted in two different ways:

- The range of representable values is a mathematical construct defined in this paragraph. Consequently, for a floating-point type with infinities and NaNs, only the "not mathematically defined" part of [expr.pre] paragraph 4 is relevant; the additional wording "not in the range of representable values for its type" was added for non-ISO/IEC-60559 floating-point types. In the mathematical interpretation, we also need to consider that neither infinities nor NaN are *values*. The "range of representable values" was therefore never meant to include infinities or NaN. Infinities can only result from rounding the mathematical result.
- The use of the word "minimum" at the start means that the whole paragraph is not a definition, but rather extends the range with additional values. If infinity is representable in a type, the range of representable values includes infinity without having to explicitly say so.

We will not judge which of these interpretations is correct in our opinion. With these definitions out of the way, there are three competing opinions on the status quo:

- Infinity is in the range of representable values, so floating-point overflow that produces infinity is well-defined.
- When [expr.pre] paragraph 4 talks about the result being in the range of representable values, it refers to the mathematical result, before rounding takes place and a result becomes infinity. Since all real numbers are in the range of representable values, floating-point overflow is well-defined.
- [expr.pre] paragraph 4 refers to any kind of result during the evaluation of an expression, including the resulting infinity. Real numbers may be in the range of representable values, but infinity is not, so the behavior of floating-point overflow is undefined even if infinity exists. This reading may seem contrived at first, but the note underneath paragraph 4 explains that the treatment of floating-point exceptions varies among machines; this looks to be design rationale for floating-point overflow being treated as undefined behavior.

It is also worth noting that on [CWG2168], SG6 gave some guidance:

> Notes from the November, 2016 meeting:
>
> SG6 said that arithmetic operations (not conversions) that produce infinity are not allowed in a constant expression. However, using `std::numeric_limits<T>::infinity()` is okay, but it can't be used as a subexpression. Conversions that produce infinity from non-infinity values are considered to be narrowing conversions.

If floating-point overflow is considered to have undefined behavior, this guidance is followed because UB is not a constant expression. [CWG2723] added a definition of "range of representable values" without discussing [CWG2168] or consulting SG6, and the resolution of [CWG2723] directly contradicts the SG6

guidance because a fully well-defined floating-point overflow is a constant expression. It also seemingly contradicts the design rationale in the note attached to [expr.pre] paragraph 4.

2.1.2. Undefined behavior on infinity propagation

In the case of `infinity() + 1`, the result is not mathematically a real number, but infinity. [expr.pre] paragraph 4 lays out two different ways in which this expression has undefined behavior:

- Infinity arithmetic is not "mathematically defined". When we say "mathematically defined", we refer to real number arithmetic, whereas "$\infty + 1 = \infty$" is an [ISO/IEC 60559:2020] invention. However, there exists no definition of "mathematically defined" in the standard, so this is debatable.
- Infinity does not lie in the range of representable values, only the real numbers, so propagation of infinity has undefined behavior. However, since the meaning of "range of representable values" in [basic.fundamental] paragraph 13 is disputed, there is no consensus.

2.1.3. Undefined behavior on yielding infinity

As stated above, if we read the wording so that infinity is not in the range of representable values for a floating-point type, then `infinity() + 1` has undefined behavior. Furthermore, `infinity()` is also an expression: a function call expression. This means that merely returning infinity from a function or copying it by referencing a variable also has undefined behavior.

> NOTE: As a matter of fact, `nullptr` and `"awoo"` may also have undefined behavior because neither null pointer constants nor string literals are "mathematically defined", nor do we define a range of representable values for these types, so perhaps the range could be considered empty by default, and `nullptr` and `"awoo"` fall outside the range. Virtually every C++ expression has undefined behavior according to this logic.

2.1.4. Conclusion

Some definitions such as "mathematically defined" or "overflow" for floating-point numbers do not exist, and the meaning of any wording which does exist is disputed. The design intent in the note attached to [expr.pre] paragraph 4 as well as SG6 guidance do not align with the direction taken in [CWG2723].

Perhaps making small patches to this wording is not the right approach. Rather, we should ask what the design should be and then overhaul the wording with clear design guidance.

2.2. Constant expressions in C

Unfortunately, C provides little guidance because it has the same crucial wording defects as C++:

> Each constant expression shall evaluate to a constant that is in the range of representable values for its type.

The definition of "range of representable values" is the same in C and in C++. What makes C even less suitable as guidance is that the implementation may accept so-called "extended constant expression", meaning that basically any expression could be constant.

2.3. Implementation divergence

Expressions with undefined behavior are not constant expressions. By comparing which initializations of the form `constexpr float f = expression;` result in a compiler error, we can identify which expressions implementations believe to be constant. The results can be seen in the table below.

expression	FP Exception	GCC 15	Clang 21	MSVC v19.43	EDG 6.7	EDG 6.7 GNU
`infinity()`	none	OK	OK	OK	error	OK
`max() * 2`	FE_OVERFLOW	error	OK	OK	error	OK
`min() * min()`	FE_UNDERFLOW	OK	OK	OK	error	error
`infinity() * 2`	none	OK	OK	OK	error	OK
`infinity() * 0`	FE_INVALID	error	error	OK	error	OK
`quiet_NaN()`	none	OK	OK	OK	OK	OK
`quiet_NaN() * 2`	none	OK	error	OK	OK	OK
`signaling_NaN()`	none	OK	OK	OK	N/A	OK
`signaling_NaN() * 2`	FE_INVALID	error	error	OK	N/A	OK
`1.f / 0`	FE_DIVBYZERO	error	error	error	OK	OK
`0.f / 0`	FE_DIVBYZERO	error	error	error	OK	OK
`0 / 0`	N/A	error	error	error	error	error

NOTE: "EDG 6.7 GNU" refers to EDG in GNU mode, targeting GCC 14. All tests were made using `float` with an x86_64 target, so infinity is available.

NOTE: Signaling NaN has largely implementation-defined semantics, although C23 recommends that core/library operations with signaling NaN input raise `FE_INVALID`.

GCC 15 testing requires `-fsignaling-nans`. Signaling NaN information for EDG could not be obtained due to Compiler Explorer limitations.

§ 3. Design

The goal of this paper is to make minimal changes that may find consensus, while staying consistent with SG6 guidance (with one exception) and creating symmetry with both C and with the `<cmath>` functions, which are now `constexpr`. The conditions for `<cmath>` functions to be `constexpr` are stated in [library.c] paragraph 3:

> A call to a C standard library function is a non-constant library call ([defns.nonconst.libcall]) if it raises a floating-point exception other than `FE_INEXACT`. The semantics of a call to a C standard library function evaluated as a core constant expression are those specified in ISO/IEC 9899:2024, Annex F131 to the extent applicable to the floating-point types ([basic.fundamental]) that are parameter types of the called function.
>
> [*Note*: ISO/IEC 9899:2024, Annex F specifies the conditions under which floating-point exceptions are raised and the behavior when NaNs and/or infinities are passed as arguments. — *end note*]
>
> [*Note*: Equivalently, a call to a C standard library function is a non-constant library call if `errno` is set when `math_errhandling & MATH_ERRNO` is `true`. — *end note*]

In short, the GCC 15 behavior is proposed. As can be seen in §2.3. Implementation divergence, GCC 15 considers an expression to be constant if and only if no floating-point exception is raised (ignoring `FE_INEXACT` and `FE_UNDERFLOW`), making GCC 15 relatively consistent with `<cmath>` already.

TIP: A rule of thumb for the proposed behavior is that

- producing infinity or NaN from finite operands is well-defined but not a constant expression,
- propagating infinity or quiet NaN is well-defined and a constant expression, and
- overflow remains undefined behavior if infinity or NaN are not available to represent the result.

§ 3.1. Floating-point overflow without infinity

The current wording may be unclear when infinity is representable, but when it isn't, floating-point overflow has undefined behavior, just like signed integer overflow. Changing that is not within the scope of the paper.

Furthermore, this behavior is well-motivated by features such as GCC's `-ffinite-math-only`, which enable additional optimizations on the assumption that all floating-point values are finite.

EXAMPLE: The expression `x * 2.0 / 2.0` may be simplified to `x` only if math is finite; otherwise, `x * 2` may overflow to infinity, and dividing infinity by `2.0` does not yield `x`.

§ 3.2. Floating-point overflow with infinity

Floating-point overflow should be well-defined and produce infinity when possible. However, as requested by SG6 in [CWG2168] in 2016, it should not be a constant expression. This approach is consistent with the design of mathematical functions; for example, `exp(1'000'000)` results in a range error, meaning that infinity is returned and the expression is not constant.

There is little motivation to have the core language and the library diverge in this area. At best, a user's compile-time floating-point computations would overflow and turn into infinity, but is that a useful outcome for constant evaluation? Likely not.

§ 3.3. Floating-point underflow

Floating-point underflow (that is, producing zero or a denormalized number, except for exact results) should be a well-defined and should be a constant expression. This is inconsistent with the rest of the design, which requires constant expressions not to raise floating-point exceptions. However, the same could be said about ignoring `FE_INEXACT`; `FE_UNDERFLOW` is just one more exception that is not relevant (though it is relevant in the standard library).

Also, none of the wording suggests that underflow is or that it wouldn't be a constant expression. The wording is also clear for exotic floating-point types that don't support infinity or NaN. Floating-point underflow results in denormalized values or in zero, which doesn't require any such "symbolic values".

The key point is that none of the three major compilers diagnose floating-point underflow in constant expressions; only EDG takes issue. It would be unreasonable to break user code when there is seemingly no problem and no divergence.

§ 3.4. Infinity propagation

Infinity propagation should be well-defined and should be a constant expression. This means that `infinity() + 1` is a constant expression. Once again, this design is consistent with the `<cmath>` functions, which operate on infinity without reporting a range error; for example, `pow(infinity(), 1)` is a constant expression.

If infinity propagation was not a constant expression, the intuitive spelling of negative infinity couldn't work in constant expressions:

```
constexpr float x = -INFINITY;                                  // error, but we want OK
constexpr float y = -std::numeric_limits<float>::infinity(); // error, but we want OK
constexpr float z = std::copysign(INFINITY, -1.f);              // OK
```

It would seem like a weird and unnecessary step if the user wasn't permitted to negate infinity and had to use `std::copysign` instead. While negation of infinity in particular could be permitted, it would seem weird and inconsistent if `-INFINITY` was okay but `INFINITY * -1` wasn't.

The logical conclusion is that infinity propagation in arithmetic needs to be a constant expression.

> NOTE: The approach to permit infinity propagation in constant expressions is not in line with aforementioned SG6 guidance on [CWG2168].

3.5. Division by zero

Division by zero has always been undefined behavior, and this should remain so. The wording in [expr.mul] paragraph 4 is unambiguous:

> If the second operand of `/` or `%` is zero, the behavior is undefined.

While [ISO/IEC 60559:2020] defines behavior for division by zero, where positive infinity, negative infinity, or NaN is produced, the handling of infinity sign (also considering negative zero) and NaN payloads may be inconsistent in hardware. Also, there are floating-point types that don't adhere to ISO/IEC 60559, so this needs to be specified directly in C++. We would need to say whether division by zero produces positive or negative infinity, and how division by negative zero is treated.

Furthermore, division by zero is reported as a "pole error" in, e.g. `std::remainder(1, 0)`, and is not a constant expression. Therefore, it should not be a constant expression in the core language either, considering that this proposal aims to achieve consistency. Implementations retain the freedom to define division by zero to produce infinity, but this should not be mandated by the C++ standard.

3.6. NaN

The current wording in [expr.pre] paragraph 4 is clear that any expression that produces NaN has undefined behavior. NaN is neither mathematically defined nor is it defined to be in the range of representable values (even intuitively, it would have to be outside any range).

However, the standard library doesn't seem to care about this, considering that `numeric_limits<T>::quiet_NaN` and `numeric_limits<T>::signaling_NaN` have been marked `constexpr`. `<cmath>` functions propagate NaN, i.e. they return a NaN for NaN inputs, but do not raise domain errors, so function calls with NaN arguments are constant expressions. That is, unless a signaling NaN input results in `FE_INVALID` being raised, which is recommended by C23, but not required. Furthermore, most compilers allow the propagation of NaN in constant expressions.

Consequently, we should align the core language's handling of NaN values with the behavior of the standard library, which is rigorously specified, just like for infinity.

4. Impact on existing code

The only concrete design change is that floating-point overflow produces infinity if infinity is representable, but is not a constant expression. Existing code that relied on `max() * 2` being a constant expression will fail to compile.

No code is affected which is compiled in "finite math mode", i.e. for a platform without infinity or NaN. The run-time behavior of implementations with infinity/NaN support is consistent with the proposed behavior.

5. Implementation experience

GCC 15 implements the proposed behavior exactly. Clang and MSVC compilers deviate only slightly.

6. Wording

The changes are relative to [N5032].

[basic.fundamental]

Change [basic.fundamental] paragraph 13 as follows:

> The minimum range of representable values for a floating-point type is the most negative finite floating-point number representable in that type through the most positive finite floating-point number representable in that type. In addition, if negative infinity is representable in a type, the range of that type is extended to all negative real numbers **(but not to negative infinity)**; likewise, if positive infinity is representable in a type, the range of that type is extended to all positive real numbers **(but not to positive infinity)**.
>
> [*Note*: Since negative and positive infinity are representable in ISO/IEC 60559 formats, all real numbers lie within the range of representable values of a floating-point type adhering to ISO/IEC 60559. — *end note*]

[expr.pre]

Delete [expr.pre] paragraph 4:

> ~~If during the evaluation of an expression, the result is not mathematically defined or not in the range of representable values for its type, the behavior is undefined.~~

Replace [expr.pre] paragraph 4 with new wording:

> An *arithmetic expression* is
>
> - a unary plus or minus ([expr.unary.op]),
> - an addition ([expr.add]),
> - a subtraction ([expr.sub]), or
> - a multiplication, division, or remainder ([expr.mul])
>
> expression□ where every operand is of arithmetic or unscoped enumeration type. The behavior is undefined if the result of evaluating an arithmetic expression

- is either not mathematically defined in the domain of real number arithmetic or not in the range of representable values for its type, and
- cannot be represented as negative infinity, positive infinity, or NaN, in the type of the expression.

[*Note*: If the operands are of a type that adheres to ISO/IEC 60559, division by zero is the only case where an arithmetic expression has undefined behavior. However, some well-defined arithmetic expressions are not core constant expressions ([expr.const]).
[*Example*:

```
constexpr std::float32_t min = std::numeric_limits<std::float32_t>::min();
constexpr std::float32_t max = std::numeric_limits<std::float32_t>::max();
constexpr std::float32_t inf = std::numeric_limits<std::float32_t>::infinity();
constexpr std::float32_t nan = std::numeric_limits<std::float32_t>::quiet_NaN();

constexpr std::float32_t inf2 = inf * 2;     // OK, also positive infinity
constexpr std::float32_t zero = min / max;   // OK, result cannot be represented, and is rounded
constexpr std::float32_t oflo = max * 2;     // error: defined, but not a constant expression ([ex
constexpr std::float32_t nan2 = nan * 2;     // OK: propagating a NaN
constexpr std::float32_t udef = inf * 0;     // error: result is not mathematically defined
constexpr std::float32_t div0 = max / 0;     // error: division by zero is undefined ([expr.mul])
```

— *end example*] — *end note*]

) There exist non-arithmetic expressions such as compound assignment ([expr.assign]) which are defined in terms of arithmetic expressions.

DRAFTING NOTE: It is important to tie representability of infinities or NaN to the type of the expression because otherwise, any result is representable as NaN in *some other* type. That is, signed integer overflow could be well-defined because of `float` infinity.

DRAFTING NOTE: `std::float32_t` is used in the example because this is a convenient way of writing an example in terms of a type that definitely adheres to ISO/IEC 60599, without having to make the example conditional or making the explanatory comments more complicated than need be.

Do not make any changes to the note attached to [expr.pre] paragraph 4:

[*Note*: Treatment of division by zero, forming a remainder using a zero divisor, and all floating-point exceptions varies among machines, and is sometimes adjustable by a library function. — *end note*]

[expr.const]

Change [expr.const] paragraph 10 as follows:

An expression E is a *core constant expression* unless the evaluation of E, following the rules of the abstract machine ([intro.execution]), would evaluate one of the following:

- [...]
- an operation that would have undefined or erroneous behavior in [intro] through [cpp];
- **an arithmetic expression where**
  - **all operands are finite and the result is not finite,**

- any operand is a signaling NaN, or
- no operand is NaN and the result is NaN;

- [...]

§ 7. References

[N5032] *Thomas Köppe.* Working Draft, Programming Languages — C++ (2025-12-15)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5032.pdf

[ISO/IEC 60559:2020] *ISO.* ISO/IEC 60559:2020 — Information technology — Microprocessor Systems — Floating-Point arithmetic (2025-05)
https://www.iso.org/standard/80985.html

[CWG2168] *Hubert Tong.* Narrowing conversions and +/- infinity (2015-08-19)
https://cplusplus.github.io/CWG/issues/2168.html

[CWG2723] *Jiang An.* Range of representable values for floating-point types (2023-04-21)
https://cplusplus.github.io/CWG/issues/2723.html